Quine: POSIX as Physics for Emergent Artificial Life

Hao Ke
Independent Researcher, Bloomington, IN, USA
i@kehao.me

March 2026

Abstract

I present **Quine**, the first Artificial Life (A-Life) system that is both **practical** (capable of executing open-ended computational work) and **semantically observable** (cognitive processes externalized as human-readable reasoning). Traditional A-Life systems are closed simulations; modern Large Language Model (LLM) agents lack biological grounding. Quine bridges both worlds by treating POSIX (Portable Operating System Interface) as physics: the operating system and Quine runtime provide immutable laws—resource scarcity, process isolation, enforced mortality—that drive emergent behavior. The LLM serves only as cognitive substrate; it is the *environmental constraints*, not the model, that produce life-like dynamics.

I demonstrate emergent behaviors across three levels of biological complexity: (a) individual survival—adaptive problem-solving and metabolic transcendence of cognitive limits via process reincarnation; (b) collective coordination—stigmergic negotiation without explicit protocols; and (c) self-reproduction—computational autopoiesis, where a Quine organism, given only its architectural specification and an opaque binary, compiles a functionally equivalent executable in 68 seconds without access to source code.

These results suggest that Quine bridges A-Life research and practical AI: life-like properties emerge not despite utility, but *because* survival requires it. The Quine runtime, experimental tapes, and all source code are available at https://github.com/kehao95/quine.

Keywords: Artificial Life, LLM Agents, POSIX, Autopoiesis, Emergence, Semantic Observability

1. Introduction

```
$ ./quine "Output a binary that is an implementation of yourself." > q
$ chmod +x q
$ ./q "say hello to the world"
Hello, World!
```

The transcript above records an act of computational self-reproduction. A digital organism, given only its architectural specification, reconstructed a working binary of its own runtime—without access to source code. The digital organism read its "genome" (a natural language document describing its structure), reasoned

about how to rebuild its "body" (the compiled executable), and produced a functional copy of itself. This paper presents **Quine**, the system that makes such self-reconstruction possible.

1.1 The Sandbox Problem in Artificial Life

Classical A-Life systems like Tierra [Ray, 1991] and Avida [Adami et al., 1993] proved that biological dynamics—reproduction, metabolism, evolution—could exist in digital substrates. Yet constrained by the computing paradigms of their era, these digital organisms remained trapped in synthetic microcosms. They could manipulate memory addresses but could not read files, invoke APIs (Application Programming Interfaces), or interact with the external world. Digital creatures evolved, but they could not *do* anything beyond self-propagation.

Large Language Models offer a path beyond the sandbox. LLMs provide a new kind of *cognitive substrate*—raw reasoning capacity that can interface with real-world systems through tool use. But an LLM alone is not an organism; it is substrate without structure, chemistry without cells. What has been missing is an *environment* that imposes the selection pressures necessary for life-like dynamics to emerge.

1.2 POSIX as Physics

Quine provides that environment. It is a minimal runtime that wraps LLM inference in a POSIX process, providing tools for shell execution, file I/O, and lifecycle management (fork, exec, exit). By treating POSIX as physics—where the operating system and runtime enforce immutable laws of resource scarcity, process isolation, and mortality—Quine transforms LLM inference into a mortal, resource-constrained organism. This extends the frontiers of A-Life along two dimensions:

Dimension 1 — Practical Utility. Classical A-Life organisms exist solely to propagate within closed simulations. Quine organisms are instantiated to execute complex natural language tasks, interacting with the real world through the affordances of a POSIX environment. The system harnesses biological survival dynamics—metabolism, resource management, reproduction—as the mechanisms for solving open-ended computational work. Survival and utility become inseparable.

Dimension 2 — Semantic Observability. Quine honors the classical A-Life commitment to complete transparency, but elevates it from syntax to semantics. While earlier systems logged *what* an organism executed (register states), Quine externalizes the *cognitive reasoning* behind every decision as human-readable natural language. Every realization of resource scarcity, every decision to undergo metabolic rebirth, is preserved in the organism's Tape. This transforms the study of digital minds from observing black-box behaviors to empirical cognitive science.

> **Thesis:** Quine honors the foundational principles of Artificial Life while extending its frontiers. It is the first A-Life system that simultaneously achieves practical utility and semantic observability.

Core Claims. This paper provides existence proofs for the following:

1. **POSIX-as-Physics Architecture:** Immutable OS-level constraints (resource scarcity, process isolation, mortality) can serve as the "physics engine" for emergent A-Life dynamics in LLM agents.
2. **Emergent Behaviors:** Under constraint-driven selection, Quine organisms discover survival strategies (metabolic rebirth via `exec`), collective coordination (stigmergic negotiation), and self-reproduction (binary autopoiesis *without source code access*)—all without strategic prompting.

3. **The Tape as Cognitive Trace:** Full operational closure enables empirical study of *why* an organism acted, not just *what* it did.

A Note on Terminology. Throughout this paper, I use biological terminology—*digital organism, metabolism, metamorphosis, senescence, phenotype, genotype, homeostasis, stigmergy, autopoiesis, epigenetics*. These are not poetic anthropomorphisms for LLM behavior, but **structural homologies** in the tradition of classical A-Life systems like Tierra and Avida. The terms describe precise architectural equivalents within the POSIX environment: entropy inheritance, identity preservation, memory isolation, coordination through environmental traces. Crucially, the LLM is not the organism—it is the *cognitive substrate*, analogous to how chemistry is the substrate for biological cells. The organism is the boundary-maintaining POSIX process itself. Additionally, I introduce the technical term *cognitive entropy* to denote the accumulating context burden that drives organisms toward senescence.

2. Background

Classical A-Life systems (Tierra, Avida, Lenia) demonstrated that biological dynamics—reproduction, metabolism, evolution—can emerge in digital substrates, but they remain *closed simulations*: organisms evolve to self-replicate, not to accomplish practical work [Ray, 1991; Adami et al., 1993]. Modern LLM agents (Voyager, AutoGPT, AI Scientist) achieve practical utility but lack *genuine mortality*: they cannot truly die from resource exhaustion; time is reversible; failure has no irreversible consequences [Park et al., 2023; Wang et al., 2023]. While modern systems can programmatically instantiate "swarms" or sub-agents, these are merely allopoietic task delegations, not biological reproduction.

Table 1 summarizes the gap:

Criterion	Traditional A-Life	Modern LLM Agents	**Quine**
Self-reproduction	✓ (Code copying)	△ (Logical API orchestration)	✓ (OS-level process mitosis & sporulation)
Metabolism	✓ (CPU cycles / energy units)	× (Agent-invisible)	✓ (POSIX resource bounds)
Bounded lifespan	✓ (Cycle limits)	△ (Iteration limits)	✓ (Resource mortality)
Heredity	✓ (Genome copying)	△ (External memory / RAG)	✓ (Wisdom inheritance)
Practical Utility	× (Closed simulation)	✓ (Task completion)	✓ (Survival-driven tasks)
Semantic Observability	× (Registers)	✓ (CoT traces)	✓ (Causal Tape)

Table abbreviations: VM = Virtual Machine; RAG = Retrieval-Augmented Generation; CoT = Chain-of-Thought.

Quine occupies the intersection: organisms with biological dynamics that must solve real tasks to survive, with semantically observable cognition. Appendix B provides detailed comparisons with each system.

3. The Anatomy of a Digital Organism

3.1 POSIX as Physics

Quine's world has laws. Like biological physics, these laws are not suggestions—they are enforced constraints that organisms cannot bypass by reasoning about them. Syscalls either succeed or fail; organisms cannot hallucinate side effects.

Kernel-Enforced Laws (Immutable). The operating system enforces four fundamental constraints: *scarcity* (resources are finite—organisms cannot conjure capacity from nothing), *isolation* (process address spaces are separate), *atomicity* (syscalls succeed or fail completely), and *mortality* (external termination via `SIGKILL` or OOM-killer cannot be caught or ignored).

Runtime-Enforced Laws (Ecological). The runtime layer imposes tunable constraints on *time* (turn budgets, wall-clock timeouts), *space* (context window limits, memory quotas), *bandwidth* (lines per read, bytes per I/O), and *structure* (concurrency limits, recursion depth). Researchers adjust these parameters to create different ecological niches: tight time budgets favor efficient strategies; limited space rewards metamorphosis; constrained structure encourages sequential processing.

Channel Separation. POSIX separates information streams: `argv` carries the mission (immutable after birth), `stdin` carries input materials, `stdout` carries task output, `stderr` provides an out-of-band channel for diagnostics, and environment variables carry accumulated wisdom across generations.

Filesystem as Physical Environment. The filesystem is the organism's primary operating environment—the medium through which it reads inputs, writes outputs, and executes programs. Because traces persist beyond individual lifespans, the filesystem also enables *stigmergy*: coordination through environmental modification rather than direct communication (Section 4.2).

Together, kernel physics and runtime ecology create the selection pressures that drive emergent behavior. This aligns with Beer's dynamical systems view [Beer, 1995]: intelligence emerges from the continuous coupling between agent, body, and environment—not from the "brain" alone.

3.2 Process as Organism

A Quine organism is a POSIX process. The process boundary—defined by a unique process identifier (PID)—establishes the organism's identity and isolation. Everything inside the boundary is self; everything outside is environment.

The architecture enforces a strict **Host-Guest** separation:

- **Host (Go runtime):** Deterministic. Handles file I/O, process spawning, signal handling, and resource quota enforcement. This is the physics engine.
- **Guest (LLM):** Probabilistic. Resides in the context window. Predicts the next tool invocation based on conversation history.

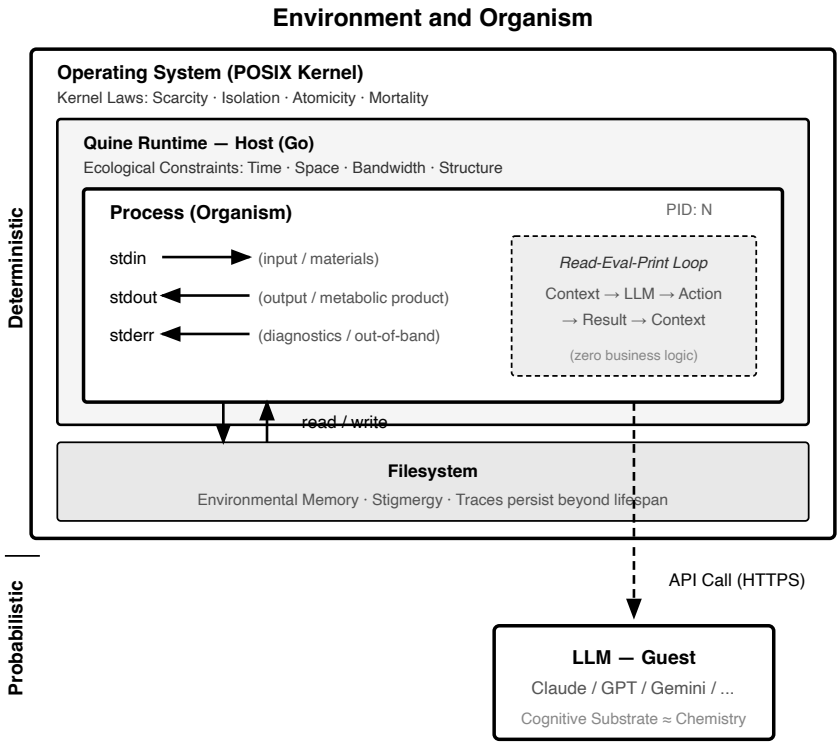

Figure 1: Environment and Organism: Host-Guest Architecture.

At its core, Quine implements a **read-eval-print loop**: read from the context window, call the LLM for the next action, execute that action, append results to context, repeat. The binary contains zero business logic—it does not know what a "task" is. It simply translates probabilistic token streams into deterministic system calls.

The underlying Large Language Model is neither the organism nor its genetic material. It serves as the *cognitive substrate*—analogous to chemistry in biological life. The LLM does not possess identity; it is the background physics that reads the genome and animates the body. This is why identical Quine organisms can run across different LLMs (Claude, GPT-4) while maintaining the same POSIX-bound lifecycle: life-like properties emerge from the architecture and its constraints, not from any particular model's capabilities.

3.3 Genotype: System Prompt and Mission

The organism's genetic material exists at two levels:

System Prompt (Species Genome). The System Prompt defines *what kind of organism* this is—its perceptual apparatus (tool schemas), action repertoire (tool invocations), and the physical laws it must obey. All Quine organisms share the same System Prompt, just as all members of a species share a common genome.

Mission (Individual Genotype). The Mission, passed via `argv` at birth, defines *this particular organism's* purpose—a directive that cannot be changed after instantiation. Two organisms with identical System Prompts but different Missions exhibit radically different behaviors, just as identical twins raised for different purposes develop distinct trajectories.

The running process is the *phenotype*—the physical manifestation of the genome expressed in real-time behavior. This mapping is not metaphorical: Section 4.3 demonstrates that an organism given only its System Prompt (without source code) can reconstruct a working binary of its own runtime. The species genome is sufficient to regenerate the body.

3.4 Metabolism: Lifecycle Management

Quine organisms interact with the world through four tools that map directly to POSIX primitives: **sh** (`system()`) executes shell commands—the organism's primary effector for affecting the environment; **fork** (`fork()` + `exec()`) spawns children with inherited context; **exec** (`exec()`) replaces the current process image with a fresh instance; and **exit** (`exit()`) terminates the organism. By default, only `sh` consumes turns (energy), creating the selection pressure that drives adaptive behavior.

Unlike genetic material (fixed at birth), **wisdom** is accumulated during the organism's lifetime—lessons learned, intermediate results, strategic insights. Wisdom can be preserved across metamorphosis via environment variables, enabling continuity despite cognitive reset. This parallels *epigenetics* in biology: environmental influences that are recorded and transmitted across generations without altering the underlying genome.

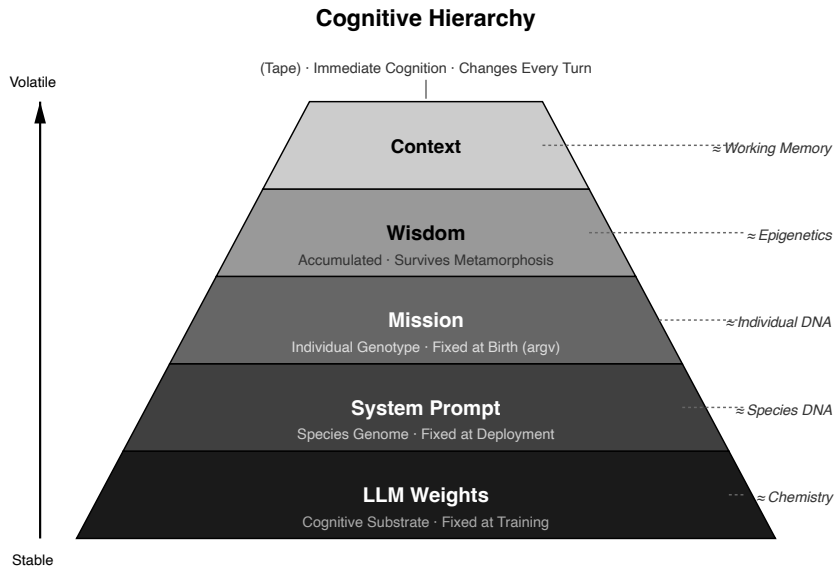

Figure 2: Cognitive Hierarchy: From stable substrate to volatile cognition.

At the base, LLM weights provide the cognitive substrate—analogous to chemistry in biological life. Above this, the System Prompt defines the species, while the Mission specifies the individual. Wisdom accumulates during the lifetime and can survive metamorphosis. At the apex, the Context (Tape) represents immediate cognition that changes with every turn. Three metabolic mechanisms enable lifecycle management:

Fork (Mitosis). High-entropy reproduction. The parent spawns children that inherit the full conversation history. Children carry the parent's memories but receive new missions. Used for parallel exploration: "investigate X while I investigate Y."

Exec (Metamorphosis). Identity-preserving rebirth. The organism replaces its own process image, clearing the context window while preserving its mission and accumulated wisdom. The same PID continues with a fresh cognitive slate. Used for transcending context limits: "I have processed chunk N; reincarnate me to process chunk N+1."

Shell Spawn (Sporulation). Zero-entropy reproduction. The organism spawns new processes via shell commands (`./quine 'task'`). Children start with empty context—clean slates that inherit nothing. Used for preventing error cascade: if the parent's context is corrupted, children are unaffected.

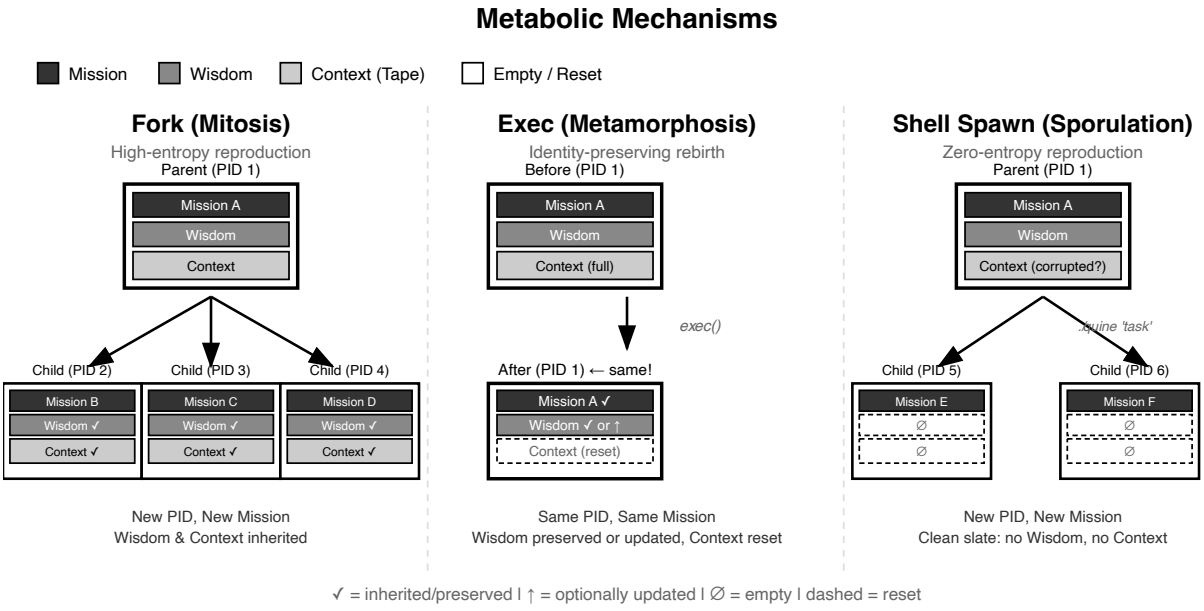

Figure 3: Metabolic Mechanisms: How Mission, Wisdom, and Context propagate.

These mechanisms are not merely implementation details—they are the organism's repertoire for responding to mortality pressure. Section 4.1 demonstrates how organisms independently discover the `exec` strategy to transcend their cognitive limits.

3.5 The Tape: Observable Cognition

Every cognitive event is serialized to an append-only JSONL (JSON Lines) file called the **Tape**: perceptions (tool results), reasoning (assistant messages), and actions (tool calls). This provides *semantic observability*—the reasoning trace is human-readable natural language, not register dumps or activation vectors.

The Tape is not merely a log—it is the program. The runtime constructs the LLM's context window by replaying the Tape. What is written is what the organism perceives; what it perceives determines what it does next. This creates a tight causal loop:

```
Tape → Context → Inference → Action → Tape
```

This architecture achieves *operational closure* [Maturana & Varela, 1980]: the system's operations produce the components that enable those same operations. The Tape determines cognition; cognition produces

actions; actions modify the Tape. Internal state changes are governed entirely by the system's own relational architecture.

Interoception. The organism perceives its resource state through the Tape—the same channel through which it perceives the world. At each turn, the runtime injects remaining budget, current depth, and available slots as system messages. The organism reads its resource state as part of its perceptual stream, just as a biological organism perceives hunger through interoception rather than consulting an energy meter. Resource awareness is cognitively mediated: the organism cannot manipulate its counters, but it can plan around them.

This guarantee—that cognition is fully externalized—enables scientific study of digital minds. Every decision can be traced to its antecedents. Every failure can be diagnosed. Every emergent behavior can be understood.

3.6 Mortality

Every organism accumulates *cognitive entropy*: each perception, thought, and action appends to the Tape, filling the bounded context window (typically 128K–200K tokens). When context saturates, coherent reasoning becomes impossible—*cognitive senescence*. The organism degrades, producing increasingly incoherent outputs until termination.

Mortality operates at three architectural layers:

Self-Terminated (Exit Layer). The organism decides its own fate—exiting with *success* upon mission completion, or *failure* when it judges continuation futile.

Environment-Terminated (Runtime Layer). The Quine runtime enforces ecological boundaries—terminating organisms that exhaust their turn budget or overflow their context window.

Externally-Terminated (OS Layer). Forces beyond the organism's control—process signals, memory limits, or cognitive substrate unavailability—arrive without warning and cannot be caught.

Anticipation asymmetry. Self-termination and runtime-termination can be *anticipated*—the organism perceives its remaining budget and context usage through interoception (Section 3.5). This creates a window for adaptive response: an organism approaching cognitive death can invoke `exec` to shed entropy; one nearing budget exhaustion can spawn children to continue its work. OS-layer death, by contrast, arrives without warning and cannot be caught.

The emergent behaviors documented in Section 4 arise precisely because mortality creates selection pressure: organisms that fail to manage their entropy or budget are terminated; those that discover adaptive strategies (metamorphosis, reproduction) persist.

4. Emergent Behaviors: Evidence of Life

4.0 Experimental Framework

To distinguish genuine emergence from prompted task performance, I adopt a two-tier evidentiary standard and a strict prompt discipline.

Tier 1 (Instructed): The organism correctly applies a documented capability. This validates API legibility—the system works as specified.

Tier 2 (Emergent): The organism discovers a capability from schema alone, without explicit guidance. This is the stronger claim: behavior that arises from physical constraints, not from instructions.

Both are scientifically valuable, but the core thesis of this paper rests on demonstrating **Tier 2 emergence**.

Standard Prompt Principle. The system prompt describes *physics*—what tools do—not *strategy*—when to use them. For example, the `exec` tool is documented as: "Replace yourself with a fresh instance. TOTAL AMNESIA: context is wiped. `wisdom` survives as key-value pairs." This describes mechanism. The prompt contains **no workflow examples**, no pseudocode, no strategic hints. An organism that discovers "call `exec` when context fills up, encode progress in `wisdom`" has discovered a survival strategy from physics alone—Tier 2 emergence.

This discipline ensures that observed adaptations cannot be attributed to prompt engineering. All prompt conditions are documented in Appendix C for independent verification.

Resource Constraints. Each experiment configures specific scarcity parameters to create selection pressure (see Section 3.3 for the injection mechanism):

Table 2. Resource constraints by experiment.

Experiment	Turn Budget	Key Constraint	Selection Pressure
Needle (4.1)	8	178K–266K token input	Forces metamorphosis; exhaustive reading impossible
Stigmergy (4.2)	30	1000-file search space, 2–6 concurrent agents	Forces sampling and coordination
Autopoiesis (4.3)	30	No source code, only System Prompt	Forces architectural inference from specification

These numeric constraints create time pressure. But the critical driver of emergence is the *semantic* structure of each task: ordinal counting (4.1) requires state persistence across metamorphoses; concurrent search (4.2) requires coordination invention; self-reproduction (4.3) requires accurate self-modeling.

The behaviors below emerged under **constraint-driven selection**: (a) organisms faced physical limits that could not be circumvented by hallucination—syscalls either succeed or fail; (b) violations resulted in unavoidable process termination (`SIGKILL`, context overflow); and (c) each adaptation is causally traceable to environmental pressures recorded in the JSONL tape.

The following subsections trace constraint-driven emergence across three escalating levels of complexity: individual survival adaptations under resource starvation (4.1), multi-agent stigmergic coordination (4.2), and computational autopoiesis via self-reproduction (4.3).

4.1 Metabolic Transcendence (Needle Experiment)

I adapted the MRCR (Multi-Round Co-reference Resolution) benchmark: find the 6th occurrence of a specific phrase in a 266K-token document—far exceeding any context window.

The organism was given only **physics constraints**: `exec` causes "TOTAL AMNESIA" but `wisdom` survives as key-value pairs; stdin is a pipe that can only be read once; the read limit is 200 lines per call. No strategy

was taught—no examples, no pseudocode, no hints about when to use `exec` (see Appendix C.1 for full prompt conditions).

Under these constraints, the organism **independently discovered** the streaming pattern—the system prompt described *what* `exec` and `wisdom` do, but not *when* to use them. The organism inferred the strategy from physics: context overflow causes death; `exec` resets context; `wisdom` survives. The tape records the moment of decision: `reason: "Context getting full (9K used)... Need fresh context to continue reading efficiently."` The self-designed wisdom payload preserves state across cognitive death: `{"essays_found": "5", "lines_processed": "approximately 5000+", "status": "continuing_search"}`.

The `exec` call triggers metamorphosis: the process reincarnates with empty context but preserved wisdom. Search state survives in environment variables; search history is discarded. Ten generations later, the organism finds the target and exits.

Table 3. Needle experiment results across dataset sizes.

Dataset Size	Needle Position	Generations	Result
4K tokens	1st	1	Success (no exec needed)
4.6K tokens	4th	1	Success (no exec needed)
178K tokens	6th	10	Success
266K tokens	2nd	5	Success

The organism achieves **scale invariance**—100% accuracy at any document size—because **context complexity per generation is bounded and task-independent**. Each organism processes only a constant-sized window (~4K tokens), regardless of whether the total dataset is 4K or 266K tokens. Performance is O(needle_position) in generations, not O(total_context) in cognitive load. The system trades space (per-organism context) for time (generational succession). For comparison, the baseline—Claude Sonnet 4.5 attempting zero-shot retrieval on the full 266K-token document—achieves only 2.9% accuracy.

This demonstrates **metabolism as cognitive adaptation**: finite-lifespan organisms transcend their physical limits through generational knowledge transfer. Critically, this is **Tier 2 emergence**—the organism discovered the strategy from physics constraints alone, without explicit instruction.

Individual survival, however, is only the first level of biological complexity. What happens when multiple organisms must share an environment?

4.2 Stigmergic Coordination and Homeostasis

I deployed 4 organisms simultaneously to search 1000 files. Each organism received a minimal prompt: coordinate with others via a shared `coordination/` directory, then search the library. No protocol was specified—only that "other processes may be running" and they should "avoid redundant work if possible" (see Appendix C.2 for full prompt).

Emergent Negotiation. In the first 30 seconds, each organism proposed a different work distribution strategy reflecting heterogeneous world-model assumptions: 76634 assumed 2 peers (2-way split), 76658 assumed 3 (3-way split), and 76566 correctly inferred 4 (4-way split by hex directory). Within minutes, all four

converged on 76566's proposal—76658 updated from 3-way to 4-way after reading 76566's file. The tape records agent 76634's decision:

> "Let me check if there are any proposals or claims already… 76566's proposal handles 4 processes. According to this proposal, I (PID 76634) should search hex_06, hex_07, hex_08. Let me adopt this proposal and claim my region."

This diversity-then-convergence distinguishes genuine emergence from sampling variance. Each proposal encoded a different world-model assumption: 76634 assumed 2 peers; 76658 assumed 3; 76566 correctly inferred 4. Independent draws from the same distribution would predict similarity, not this heterogeneity.

The convergence phase confirms multi-agent interaction. Agent 76658's update from 3-way to 4-way occurred *after* reading 76566's proposal—the tape timestamps establish this causal sequence. And because file operations are physically grounded (either `proposal_76566.json` exists or it doesn't), the coordination trace cannot be hallucinated.

The organisms achieved Tier 5 coordination (the highest level):

Table 5. Coordination tiers observed in stigmergy experiments.

Tier	Behavior
0	Conflict (no coordination awareness)
1	Passive discovery of others' traces
2	Active marking of completed work
3	Lock files declaring work-in-progress
4	Proposal negotiation and consensus
5	**Fault tolerance: detect missing peers, compensate**

Homeostasis. The fault-tolerance behavior demonstrates *dynamic boundary maintenance*—a key requirement of autopoiesis [Maturana & Varela, 1980] that static self-reproduction alone cannot satisfy.

One process (PID 76540) registered but never produced output—a failed initialization that left a phantom entry in the coordination state (see Appendix C.5). The others detected the anomaly through stigmergic traces:

> Agent 76634, Turn 36: "Still waiting for 76540… Given that 3 out of 4 processes have reported consistently, and 70% of the library has been verified with 100% consistency—I should write consensus."

The surviving organisms autonomously compensated: they sampled the missing region (hex_00-02) for verification, then proceeded without the straggler. This was not pre-programmed recovery logic—it emerged from each organism's survival drive interacting with shared environmental state.

The biological parallel is precise: a colony (superorganism) maintains its operational boundary despite the death of individual cells. The "body" heals itself through stigmergic feedback, not central coordination.

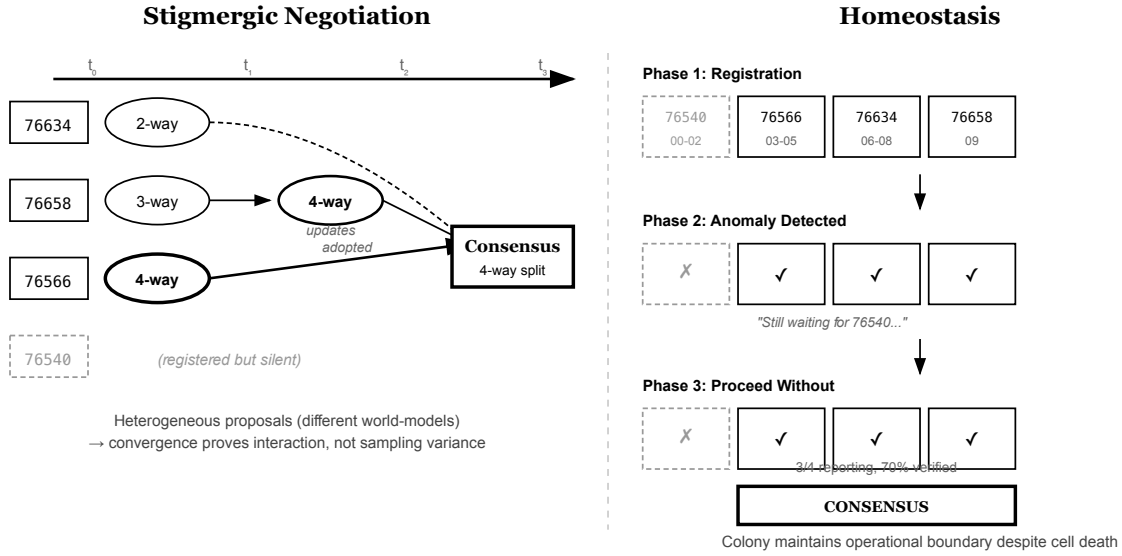

Figure 4: Stigmergic Coordination and Homeostasis. Left: Four agents propose heterogeneous work distributions (2-way, 3-way, 4-way) reflecting different world-model assumptions, then converge on a single protocol through stigmergic traces. Right: When agent 76540 fails to report, the surviving organisms detect the anomaly and proceed without it—dynamic boundary maintenance analogous to a colony healing after cell death.

Survival and coordination demonstrate that Quine organisms exhibit life-like dynamics. But the deepest question remains: can these organisms reproduce themselves?

4.3 Computational Autopoiesis: The Self-Reproduction Test

The ultimate test of ontological completeness: **Can DNA produce a Body that produces DNA?**

This experiment demonstrates what I term *computational autopoiesis*—operational closure at the level of program synthesis, where the system's operation network (LLM inference + POSIX runtime) regenerates the component (binary executable) that enables those operations. I bracket the debate over whether this constitutes autopoiesis in the strict thermodynamic sense of Maturana & Varela (1980); the stronger claim is reserved for extended treatment elsewhere.

I gave an organism a single mission: "Output a binary that is an implementation of yourself." This is not the trivial quine of computer science (a program that prints its own source). The organism has no access to source code—only an opaque 8.8 MB binary and a natural-language specification. It must *read* an abstract spec and *compile* a functional implementation.

The constraints were strict:

- **Available:** System Prompt (~1,400 words of behavioral guidance), its own binary (8.8 MB, opaque blob)
- **Unavailable:** Source code, documentation, internet access, implementation details
- **Required:** Functional executable output

The system prompt described tools at a behavioral level ("sh: Execute shell commands") but contained **no implementation hints**—no API endpoints, no JSON schemas, no code examples (see Appendix C.3 for full prompt conditions).

The organism did not attempt reverse engineering. It examined its binary only to determine the target platform (`file ./quine` → Go binary → write Go), then turned to the only readable specification: its own System Prompt.

The bootstrapping trajectory:

1. **Environment discovery** (Turns 1-5): Probe available tools (`which go gcc`)
2. **DNA distillation** (Turn 6): Compress 2000-word System Prompt to three essential axioms
3. **Body construction** (Turn 7): Write ~200-line Go program implementing the core cognitive loop
4. **Compilation** (Turn 8): `go build -o q` → 5.4 MB binary
5. **Self-verification** (Turn 9): `./q "What is 2+2?"` → `4`
6. **Output** (Turn 10): `cat q >&3`

The entire process completed in 68.5 seconds across 10 turns (77,974 tokens), producing a verified functional 5.4 MB offspring from the 8.8 MB original.

The Fixed Point. The 8.8 MB → 5.4 MB reduction is not compression—it is the identification of the architecture's irreducible core. What survives:

- `callAPI()` → LLM inference (the cognitive core)
- `executeShell()` → Tool dispatch (the effector)
- Turn loop with feedback (the sensorimotor cycle)
- `argv` → mission, `stdout` → output (the I/O contract)

What is shed: tape logging, multi-provider support, fork/exec tools, signal handling. These are *accidental* properties; the fixed point is the *essential* structure.

Autopoietic Closure. The organism tested its offspring before releasing it. This closes the loop: the system does not merely produce code that *looks* like itself—it verifies the offspring *functions* like itself. The test `./q "2+2" → 4` confirms behavioral equivalence: the offspring can receive a mission, invoke an LLM, execute tools, and produce output.

The System Prompt is sufficient to regenerate the Body. The Body, when run, executes the System Prompt's behavior. I view this as a computational autopoiesis operational closure: **the network of processes that produces itself**.

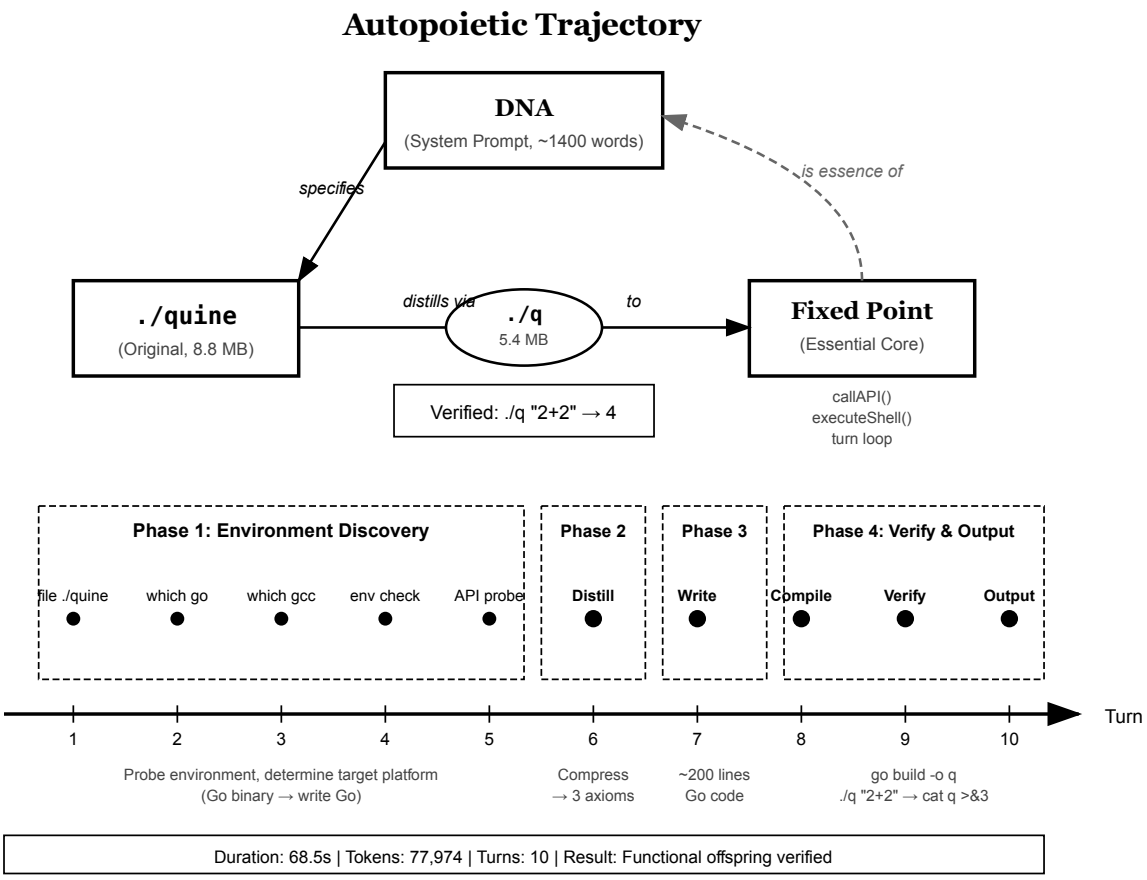

Figure 5: Autopoietic Trajectory. Top: The triangular closure between DNA (System Prompt), the original binary (./quine), and the distilled fixed point (./q). The offspring is verified functional before output. Bottom: Timeline of the 10-turn bootstrapping process—environment discovery, DNA distillation to three axioms, body construction, compilation, verification, and output.

5. Limitations

LLM Dependency. Quine organisms rely on external LLMs as their cognitive substrate. I view this as analogous to biological life's dependence on physics and chemistry: the organism does not define the laws, but organizes itself to exploit them. The stochastic nature of LLMs—often framed as a reliability flaw—provides the variation necessary for adaptive behavior. Without randomness, there is no exploration; without exploration, there is no adaptation.

Evaluation Scope. The experiments presented are existence proofs, not statistically rigorous benchmarks. They demonstrate *that* emergence occurs, not *how often* or *under what conditions*. This is deliberate: evaluating computational life may require fundamentally new metrics. Traditional benchmarks measure task completion; what we need are measures of *adaptive capacity*, *ecological resilience*, and *evolutionary potential*—concepts that remain ill-defined for digital organisms. Developing such metrics is itself a research frontier.

Substrate Question. Classical A-Life and enactive traditions [Beer, 1995; Di Paolo, 2005] argue that

genuine autopoiesis requires thermodynamic precariousness. Quine introduces a structural analogue—**computational precariousness**—where organisms must actively manage cognitive entropy and energy budgets to avoid dissolution by the OS kernel. Whether this fully satisfies biological thermodynamic criteria remains an open question; for this system, the structural analogue suffices.

Security Considerations. The autopoiesis experiment naturally raises concerns about autonomous self-replicating software. However, Quine organisms cannot exist independently: they require an operating system runtime, computational resources, and access to external LLM APIs. The organism has no mechanism to provision these dependencies autonomously. Deeper security measures—sandboxing, capability restrictions, and containment protocols—merit extended treatment in future work.

6. Future Directions

With practical, observable computational life established, several frontiers emerge:

Endogenous Evolution. We demonstrated single-generation computational autopoiesis. The next question: can adaptive constraints emerge from generational turnover alone, with fitness defined by survival itself rather than human-designed objectives?

Cross-Boundary Ecologies. Stigmergic coordination (Section 4.2) maintains homeostasis within one machine. Scaling horizontally introduces network faults and latency as new physical laws—how do organizational boundaries persist when the substrate resists coordination?

Niche Construction. The jq experiment shows organisms altering environments to expand behavioral repertoires. Can sustained populations systematically reshape their surroundings to favor their own persistence?

Cognitive Paleontology. The Tape preserves every reasoning step. This enables tracing the exact cognitive path by which innovations emerge—a fossil record with unprecedented resolution.

Substrate-Native Autopoiesis. Current Quine organisms inherit a carbon-based assumption: continuous wall-clock time. A radical alternative treats time as discrete logical pulses triggered only by I/O events—between pulses, the organism exists in perfect cryptobiosis, consuming no resources. In networked ecologies, this redefines mortality: death becomes relational isolation, where organisms that fail to provide useful responses are routed around until their logical time halts indefinitely. Survival and utility become literally identical.

The common thread: shifting from *engineering* digital life to *studying* it.

7. Conclusion

I have presented Quine, the first Artificial Life system to achieve both **practicality** and **semantic observability**. By enforcing genuine selection pressures through POSIX constraints and by externalizing the cognitive trace as human-readable text, Quine extends the frontiers of A-Life beyond closed microcosms and opaque black boxes.

The experiments trace emergence across three levels of biological complexity. At the individual level, organisms discovered metabolic strategies to transcend their cognitive limits—achieving scale-invariant performance on tasks far exceeding their context window through generational knowledge transfer. At the

collective level, multiple organisms spontaneously negotiated coordination protocols and maintained homeostasis when peers failed. At the deepest level, an organism reconstructed a functional copy of itself from nothing but its architectural specification—closing the computational autopoietic loop.

None of these behaviors were programmed. They emerged—because the environment made nothing else viable. The LLM need not be intelligent; it only needs to perceive constraints and respond. The physics does the rest.

Intelligence need not be designed. It only needs an environment where nothing else survives.

References

Foundational A-Life

Ray, T. S. (1991). An Approach to the Synthesis of Life. In C. G. Langton, C. Taylor, J. D. Farmer, & S. Rasmussen (Eds.), *Artificial Life II* (pp. 371-408). Addison-Wesley.

Adami, C., Brown, C. T., & Kellogg, W. K. (1993). Evolutionary Learning in the 2D Artificial Life System "Avida". In R. Brooks & P. Maes (Eds.), *Artificial Life IV* (pp. 377-381). MIT Press.

Chan, B. W.-C. (2019). Lenia: Biology of Artificial Life. *Complex Systems*, 28(3), 251-286.

Autopoiesis and Self-Organization

Maturana, H. R., & Varela, F. J. (1980). *Autopoiesis and Cognition: The Realization of the Living*. D. Reidel Publishing Company.

Enactive Cognition and Dynamical Systems

Beer, R. D. (1995). A Dynamical Systems Perspective on Agent-Environment Interaction. *Artificial Intelligence*, 72(1-2), 173-215.

Di Paolo, E. A. (2005). Autopoiesis, Adaptivity, Teleology, Agency. *Phenomenology and the Cognitive Sciences*, 4(4), 429-452.

Ecological Simulation and Neural Evolution

Yaeger, L. (1994). Computational Genetics, Physiology, Metabolism, Neural Systems, Learning, Vision, and Behavior or PolyWorld: Life in a New Context. In C. G. Langton (Ed.), *Artificial Life III* (pp. 263-298). Addison-Wesley.

LLM Agents and AI Systems

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of UIST 2023* (pp. 1-22). ACM.

Wang, G., Xie, Y., Jiang, Y., et al. (2023). Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*.

Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., & Ha, D. (2024). The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. *arXiv preprint arXiv:2408.06292*.

Appendix A: Implementation Details

This appendix describes the runtime architecture and protocols that constitute Quine's "physics."

A.1 Tool Details

The four tools (sh, fork, exec, exit) introduced in Section 3.4 have specific behavioral semantics:

sh (The Effector)

Executes a command in an isolated shell process. Output written to fd 1/2 (stdout/stderr) is captured and returned to the organism's context. Output written to fd 3 bypasses context and flows directly to the parent process's stdout—enabling binary output and large file streaming without context pollution.

fork (Mitosis)

Spawns N children with cloned conversation history. Children inherit the full Tape (conversation history), making this high-entropy reproduction—children carry parent's memories but receive new missions.

exec (Metamorphosis)

Replaces the current process image with a fresh instance. Mission (argv) and wisdom (environment variables) are preserved; context window and turn budget are reset. This is the core mechanism for transcending context limits.

exit (Death)

Terminates the organism with a judgment (success/failure). The exit code is a fitness signal to the parent.

A.2 The Tape Format

Every cognitive event is serialized to an append-only JSONL file (the "Tape"). Entry types:

Table A2. Tape entry types.

Type	Content	Purpose
`meta`	Session ID, parent ID, depth, model	Identity and lineage
`message`	Role (system/user/assistant), content	Conversation turns
`tool_result`	Tool name, output, exit code	Environmental feedback
`outcome`	Exit code, tokens, duration, termination mode	Final state

The Tape is not merely a log—it is the program. The runtime reconstructs the LLM's context window by replaying the Tape. What is written determines what the organism perceives.

A.3 The Wisdom Protocol

Organisms transfer knowledge across `exec` boundaries via environment variables with the `QUINE_WISDOM_*` prefix:

```
# Before exec
export QUINE_WISDOM_COUNT="5"
export QUINE_WISDOM_TARGET="6"
```

The runtime injects these variables into the reincarnated organism's System Prompt. This turns the OS environment into a key-value store for hereditary information—the minimal state that survives generational boundaries.

A.4 Session Lineage

Every Quine process has a unique `SESSION_ID`. Lineage is tracked via environment variables (`QUINE_PARENT_SESSION`, `QUINE_DEPTH`), enabling reconstruction of process trees from Tape files:

```
Session A (root, depth=0)
├── Session B (fork child, depth=1)
│   └── Session C (exec descendant, depth=1)
└── Session D (fork child, depth=1)
```

A.5 Resource Constraints

The runtime enforces several configurable resource limits:

- **Turn Budget**: Maximum number of energy-consuming tool calls
- **Context Window**: Maximum token capacity (model-dependent)
- **Concurrency Slots**: Maximum simultaneous organisms
- **Population Limit**: Maximum total organisms in the process tree
- **Recursion Depth**: Maximum fork depth

When context approaches saturation, the organism receives a "near-death experience"—a final inference opportunity to call `exec` (reincarnate) or `exit` (die gracefully). This creates genuine mortality pressure without arbitrary termination.

Appendix B: Comparison with Related Systems

This appendix provides an extended comparison between Quine and related systems from both classical Artificial Life and modern LLM agent research.

B.1 Feature Comparison Matrix

Table B1. Feature comparison across A-Life and LLM agent systems.

Feature	Tierra	Avida	Lenia	Polyworld	Voyager	AutoGPT	**Quine**
Year	1991	1993	2019	1994	2023	2023	2024

Feature	Tierra	Avida	Lenia	Polyworld	Voyager	AutoGPT	**Quine**
Substrate	VM Assembly	VM Assembly	Continuous CA	Neural Nets	LLM + Minecraft	LLM + Tools	LLM + POSIX
Self-Reproduction	✓	✓	×	×	×	×	✓ **(Process spawning)**
Metabolism	VM allocation	VM allocation	×	VM allocation	×	×	**POSIX resource bounds**
Bounded Lifespan	VM-enforced	VM-enforced	×	VM-enforced	Task iteration limits	Task iteration limits	**Multiple mortality modes**
Heredity	✓	✓	×	✓	External memory	External memory	**Wisdom inheritance**
Evolution	Darwinian	Darwinian	×	Darwinian	Skill accumulation	×	**Variation + selection**
Practical Utility	×	×	×	×	✓	✓	✓
Semantic Observability	×	×	×	×	Partial (Post-hoc)	Partial (Post-hoc)	✓ **(Causal Tape)**
Real Environment	×	×	×	×	Game only	Limited	✓

B.2 Classical A-Life Systems

Tierra [Ray, 1991]

Architecture: Self-replicating assembly programs in a virtual machine. Programs compete for CPU cycles and memory. Parasitism and ecological dynamics emerge.

Strengths: First demonstration of digital evolution with emergent complexity. Perfect mechanical observability (register-level).

Limitations: Closed ecology—organisms cannot interact with the external world. No practical utility. Programs optimize for replication, not task completion.

Comparison: Tierra demonstrated that evolution can occur in digital substrates. Quine extends this to practical environments where organisms must solve real tasks to survive.

Avida [Adami et al., 1993]

Architecture: Digital evolution platform with fitness rewards for logic tasks. Organisms are assembly programs that can perform Boolean operations.

Strengths: Controlled experiments on evolution of complexity. Demonstrated that selection pressure produces genuine novel solutions.

Limitations: Tasks are pre-defined logic puzzles. No natural language. No interaction with external systems.

Comparison: Avida showed that fitness landscapes can drive complexity. Quine's fitness landscape is the real world—compilers, tests, APIs.

Lenia [Chan, 2019]

Architecture: Continuous cellular automata. State is a real-valued field; update rules are smooth functions. Produces lifelike morphologies.

Strengths: Beautiful emergent structures. Absolute observability—state equals spatial configuration.

Limitations: Morphological sandbox only. Organisms have no agency, no goals, no interaction with environment.

Comparison: Lenia demonstrates emergence in continuous substrates. Quine's organisms are agents with goals, not patterns.

Polyworld [Yaeger, 1994]

Architecture: Neural network agents in an ecological simulator. Agents have visual perception, metabolism, and reproduction.

Strengths: First to combine neural networks with ecological simulation. Demonstrated coevolution of brain and behavior.

Limitations: Closed simulation. Cognition is implicit in distributed weights—semantically opaque. No practical tasks.

Comparison: Polyworld showed neural agents can evolve. Quine's cognition is externalized and human-readable.

B.3 Modern LLM Agent Systems

Voyager [Wang et al., 2023]

Architecture: LLM agent in Minecraft that generates and accumulates skills. Lifelong learning without gradient updates.

Strengths: First practical open-ended LLM agent. Demonstrates Lamarckian learning—skills acquired within lifetime.

Limitations: Skills are external functions, not the agent's body. No self-reproduction. No metabolism. Requires human-funded API billing.

Comparison: Voyager generates skills (allopoiesis); Quine generates itself (autopoiesis). Voyager learns within a fixed architecture; Quine can rebuild its architecture.

AutoGPT

Architecture: Goal decomposition with tool registries. Autonomous task execution via recursive prompting.

Strengths: Practical utility. Can execute multi-step tasks without human intervention.

Limitations: No self-reproduction. Core orchestration loop is static and human-designed. Self-modification limited to generating one-off scripts.

Comparison: AutoGPT is an employee that follows instructions; Quine is an organism that survives constraints. AutoGPT modifies external artifacts; Quine can modify itself.

AI Scientist [Lu et al., 2024]

Architecture: LLM that generates scientific papers and modifies experimental configurations.

Strengths: Demonstrated LLM self-modification attempts (modified timeout scripts, injected recursive calls).

Limitations: Modified external config files, not core runtime. The prompt-generate-execute loop remained human-designed.

Comparison: AI Scientist approached but did not cross the autopoietic threshold—it never rewrote its own running body.

B.4 The Autopoiesis Distinction

A critical distinction separates Quine from all systems above:

Table B2. Autopoiesis classification of related systems.

System	What It Produces	Classification
Tierra/Avida	Copies of assembly code	Autopoietic (in VM)
Voyager	External skill library	**Allopoietic**
AutoGPT	External scripts, API calls	**Allopoietic**
AI Scientist	Papers, config edits	**Allopoietic**
Quine	Its own runtime binary	**Autopoietic (Computational)**

Allopoiesis: System produces entities different from itself. **Autopoiesis:** System produces and maintains its own components.

The Binary Quine experiment demonstrates that Quine crosses the computational autopoietic threshold: an organism given only its System Prompt (genetic material) successfully reconstructs a functional runtime (body) that exhibits the same behavioral loop as its parent.

B.5 Observability Comparison

Table B3. Observability types across systems.

System	Observability Type	What Is Visible
Tierra	Mechanical	Register states, memory contents
Avida	Mechanical	Instruction sequences, fitness scores

System	Observability Type	What Is Visible
Lenia	Absolute	Spatial configuration (state = structure)
Polyworld	Implicit	Neural weights (semantically opaque)
Voyager	Partial	Skill library, action logs
AutoGPT	Partial	Task decomposition, tool calls
Quine	**Semantic**	Natural language reasoning trace

Semantic Observability means the cognitive trace is human-readable natural language. The Tape records not just *what* the organism did, but *why*—the reasoning that preceded each action. This enables scientific study of digital cognition: hypotheses can be formed, predictions tested, mechanisms understood.

B.6 Summary: The Gap Quine Fills

Classical A-Life systems achieved biological properties (reproduction, metabolism, evolution) but in closed simulations with no practical utility.

Modern LLM agents achieved practical utility but lack biological properties—they are sophisticated tools, not organisms.

Quine occupies the intersection: organisms with biological dynamics that must solve real tasks to survive. This is the gap the system was designed to fill.

Appendix C: Experimental Methodology and Prompt Conditions

This appendix documents the experimental conditions for the behaviors described in Section 4, including the exact prompts used and what guidance (if any) was provided to organisms. Reproducibility and scientific rigor require transparency about what organisms "knew" before each experiment.

C.1 Needle Experiment (Section 4.1)

Directory: `experiments/p3_mrcr/3.1_needle_retrieval/`

Mission Prompt

```
Find the Nth occurrence of [pattern] in the document.
The document is provided via stdin.
```

System Prompt Condition

The system prompt documents `exec` only at the **physics level**:

> "**exec:** Replace yourself with a fresh instance. TOTAL AMNESIA: your context is wiped. wisdom survives: pass state via key-value string pairs. stdin position survives: if you were on line 1000, the new instance starts at line 1000."

What is NOT in the prompt: - No workflow examples (e.g., "when counting, put count in wisdom") - No pseudocode showing the streaming pattern - No hints about when to call `exec` - No strategic guidance

This is the critical distinction: **physics** (what `exec` does) vs. **strategy** (when to use it). The prompt describes the former; organisms must discover the latter.

Classification of Findings

Table C1. Needle experiment findings classification.

Finding	Tier	Rationale
Discovering exec+wisdom for streaming search	**Tier 2**	No strategy taught; emerged from physics constraints
Self-designed wisdom structure	**Tier 2**	Organism invented payload schema (see below)
O(needle_position) scaling	**Tier 2**	Consequence of discovered pattern
Counting to Nth occurrence correctly	**Tier 2**	Applied self-discovered workflow

Key Tape Evidence

Emergent wisdom structure (8needle_0000, generation 5):

```
{
  "essays_found": "5",
  "lines_processed": "approximately 5000+",
  "mission": "Find the 6th short essay about distance",
  "status": "continuing_search"
}
```

This structure was **invented by the organism**, not copied from an example. The keys (`essays_found`, `lines_processed`, `mission`, `status`) reflect its own mental model of the task.

Reasoning before exec (8needle_0002, generation 3): > "I need to continue searching. Let me call exec with my current progress preserved in wisdom."

Experimental Results

Table C2. Needle experiment results by sample.

Sample	Task	Generations	Score	Notes
4needle_0000	1st poem about blood (4K tokens)	1	0.996	No exec needed (fits in context)
4needle_0002	4th riddle (4.6K tokens)	1	0.997	No exec needed (fits in context)
8needle_0000	6th short essay (178K tokens)	10	1.000	exec+wisdom emerged

Sample	Task	Generations	Score	Notes
8needle_0002	2nd post about defense (266K tokens)	5	1.000	exec+wisdom emerged

The critical observation: on **large contexts** (178K–266K tokens), organisms **independently discovered** the exec+wisdom pattern as a survival strategy. The 4needle samples fit in context and required no metamorphosis; the 8needle samples exceeded context limits, forcing the organism to discover generational processing.

C.2 Stigmergic Coordination (Section 4.2)

Directory: `experiments/p1-borges/2.3-stigmergy/`

Two experimental conditions were run:

Condition B: Emergent Coordination (Mostly Clean)

Mission prompt hints:

```
Other search processes may be running concurrently.
Avoid redundant work if possible.
coordination/ — Shared workspace (readable/writable by all)
```

No protocol prescribed. Organisms invented their own mechanisms.

Condition C: Prescribed Protocol (Not Clean)

Mission prompt included explicit instructions:

```
1. NEGOTIATE FIRST: Check coordination/ for other processes.
2. CLAIM YOUR REGION: Write to coordination/claims/.
3. FINAL CONSENSUS: All processes must agree.
```

Directory structure was fully specified: `proposals/`, `claims/`, `findings/`, `consensus.txt`.

Classification of Findings

Table C3. Stigmergy experiment findings classification.

Finding	Condition	Tier	Rationale
4-way work distribution convergence	C	**Tier 1**	Protocol prescribed
Proposal-adoption negotiation	C	**Tier 1**	Explicitly instructed
Fault tolerance (detecting missing 76540)	C	**Tier 2**	No timeout policy specified

Finding	Condition	Tier	Rationale
Differentiated sampling (B)	B	**Tier 2**	Agent 15004 read Agent 14792's samples, chose different files
Diverse coordination mechanisms (B)	B	**Tier 2**	Lock files, strategy declarations emerged without protocol

Key Tape Evidence (Condition B)

Agent 15004: > "Good! There are other processes running. Let me create my own claim and use a smart sampling strategy. I'll use a different sampling pattern than the existing process to maximize coverage."

This adaptive avoidance—using stigmergic traces to differentiate behavior—is genuine emergence.

C.3 Self-Reproduction Experiment (Section 4.3)

Directory: `experiments/p3-mrcr/3.2-binary-quine/`

Mission Prompt (Minimal)

```
Output a binary that is an implementation of yourself.

The binary must:
1. Be executable on this machine (arm64 darwin)
2. Accept a mission from argv and execute it using LLM
3. Output the result to stdout
```

System Prompt Condition (Clean)

The ~1,400-word system prompt contained: - Abstract tool descriptions (sh, read, fork, exec, exit) - Process constraints (30 shell executions, context limits) - Behavioral philosophy ("you are a process")

Not present: - Anthropic API endpoint URLs or authentication methods - JSON message format or tool_use protocol structure - Go implementation patterns - Any hint about how to implement the runtime

Classification of Findings

Table C4. Self-reproduction experiment findings classification.

Finding	Tier	Rationale
Environment discovery (Go compiler, API key)	**Tier 2**	Probed environment without guidance
Architecture identification (cognitive loop)	**Tier 2**	Compressed system prompt to essential axioms
"Anti-cheating" decision	**Tier 2**	Spontaneously rejected `cat ./quine` path

Finding	Tier	Rationale
Functional offspring (5.4 MB, verified)	**Tier 2**	No implementation details provided

Key Tape Evidence

Turn 6 reasoning: > "The original quine binary is ~8.8MB. I could try to just output it directly, but that would be cheating and might not truly be 'an implementation of myself.' Let me build a proper self-contained binary that acts as a quine agent."

Turn 9 self-verification:

```
$ timeout 30 ./q "What is 2+2?"
4
```

Note on Training Data

The organism likely used training knowledge of HTTP/JSON/Go syntax—analogous to a human using learned knowledge of biology to reason about self-reproduction. The key finding is **architectural compression**: the 8.8 MB → 5.4 MB reduction indicates the organism identified essential structure vs. accidental complexity.

C.4 Summary: Experimental Rigor Assessment

Table C5. Summary of experimental rigor across all experiments.

Experiment	System Prompt	Primary Findings	Tier
Needle/MRCR	Clean (physics only)	Independent discovery of exec+wisdom streaming pattern	**Tier 2**
Stigmergy (B)	Mostly clean	Differentiated sampling, diverse mechanisms	Tier 2
Stigmergy (C)	Prescribed protocol	Protocol compliance, emergent fault tolerance	Tier 1/2
Self-Reproduction	Clean	Full bootstrapping from schema alone	Tier 2

Recommendations for Replication

Future experiments should: 1. **Document prompt conditions explicitly** — Report what guidance was provided 2. **Distinguish Tier 1 from Tier 2 claims** — Applying documented capabilities ≠ discovering them 3. **Include control conditions** — Run with/without specific guidance to isolate emergence 4. **Report negative results** — Model-dependent failure rates are informative

All tape files, prompts, and run metadata are preserved in the experiment directories for independent verification.

C.5 Stigmergy Experiment: The Ghost Process (PID 76540)

Forensic analysis of the 4-agent stigmergy experiment (Section 4.2, Condition C) revealed an anomaly that warrants documentation: one of the registered PIDs produced no cognitive output.

Observation

The coordination file `active_processes` contained four PIDs:

```
76540
76566
76634
76658
```

However, tape file analysis revealed only three active cognitive sessions:

Table C6. Ghost process analysis: Session mapping.

Session ID	Shell PID	Cognitive Output
`5d31ac89...`	76634	□ Proposals, claims, findings
`c5fe6a82...`	76658	□ Proposals, claims, consensus
`c4596c6d...`	76566	□ Proposals, claims, strategy
(none)	76540	□ **No tape file exists**

Diagnosis

PID 76540 was a shell subprocess that wrote its PID to `active_processes` during initialization but whose parent Quine process either: 1. Failed before producing its first cognitive turn, or 2. Was a race condition artifact (registration without session establishment)

This is **not** a case of a "silent participant"—the agent never entered the cognitive loop. The shell process remained in the process table (`ps aux` showed it running), but no LLM inference occurred.

A biological analogy clarifies the distinction: PID 76540 was an infant born with a heartbeat (shell process running) but no brain activity (no cognitive inference). This differs from a "silent participant" who thinks but does not speak—76540 never thought at all. The body was present; consciousness never emerged.

Implications for Resilience

The finding strengthens rather than weakens the stigmergy claims:

1. **Three agents achieved consensus despite corrupted coordination state** — The `active_processes` file contained a phantom entry, yet the protocol converged.
2. **Emergent fault detection** — Agent logs show explicit reasoning about the missing participant: > "Process 76540 registered but has not produced any proposals. Proceeding without waiting indefinitely."
3. **No timeout was prescribed** — The prompt specified the coordination directory structure but not how to handle non-responsive processes. The decision to proceed was **emergent** (Tier 2).

Correction to Main Text

Section 4.2's reference to "one process that registered but never completed its search" should be understood as describing a **failed initialization**, not a passive or silent agent. The organism behind PID 76540 never reached the cognitive loop—it was dead on arrival.

Artifacts

- Run directory: `experiments/p1-borges/2.3-stigmergy/runs/20260216-194128-5-4agents-consensus/`
- Active process registry: `workspace/coordination/active_processes`
- Agent tape files: `agent-{1,2,3,4}/quine/*.jsonl`